

音乐歌单个性化管理系统详细设计说明书

1 概述

1.1 项目简介

音乐歌单个性化管理系统是一套基于前后端分离架构的个人音乐资源管理平台。系统实现用户账号权限管理、自定义歌单维护、本地音乐上传收纳、音乐资源浏览与个性化智能推荐等完整业务功能。本详细设计说明书主要阐述系统各模块的内部实现逻辑、接口设计、数据结构、页面逻辑与核心算法，为编码实现与系统测试提供精准依据。

1.2 设计目标

- 实现稳定、完整的音乐与歌单管理核心业务逻辑
- 统一接口规范、数据格式、权限校验机制
- 保证代码模块化、低耦合，便于调试、维护与扩展
- 实现基础个性化推荐能力，提升系统智能化体验

2 系统运行环境与技术栈

2.1 前端技术详情

- 开发框架：Vite + React，极速构建、热更新友好
- 路由控制：React Router 实现页面权限拦截与跳转
- 样式框架：Tailwind CSS 统一全局样式风格
- 网络请求：原生 Fetch 完成 GET/POST/PUT/DELETE 请求封装

2.2 后端技术详情

- 服务框架：Express.js 轻量后端服务

- 数据库：MongoDB 非关系型数据库，搭配 Mongoose 建模约束
- 身份认证：JWT 无状态令牌登录校验
- 文件处理：Multer 解析、接收、存储音频文件

3 数据库详细设计

3.1 用户数据表 (User)

用于存储系统所有注册用户信息，用于登录与权限校验。

- username: 字符串，唯一，用户名
- email: 字符串，唯一，用户邮箱
- password: 字符串，加密存储密码
- createTime: 时间戳，账号创建时间

3.2 歌单数据表 (Playlist)

存储用户自定义歌单数据，与用户一对多关联。

- name: 字符串，歌单名称
- desc: 字符串，歌单描述（可为空）
- userId: 关联用户 ID，外键绑定创建者
- createTime: 歌单创建时间

3.3 音乐数据表 (Music)

存储所有上传音乐资源信息。

- title: 音乐名称
- artist: 歌手/艺术家
- album: 专辑名称
- duration: 音乐时长
- filePath: 服务器存储路径
- uploadTime: 上传时间

4 模块详细设计与内部逻辑

4.1 用户认证模块详细设计

4.1.1 注册功能逻辑

前端接收用户名、邮箱、密码，完成前端非空校验；后端接收参数后，先查询用户名、邮箱是否重复，重复则返回错误提示；无重复则对密码进行加密处理，写入用户数据表，生成 JWT 令牌返回前端，前端本地存储令牌完成登录状态保持。

4.1.2 登录功能逻辑

前端提交用户名与密码，后端根据用户名查询用户数据，比对加密密码；校验成功生成有效时长 Token，返回用户信息与令牌；后续所有私有接口请求头部携带 Token 完成身份认证。

4.2 歌单管理模块详细设计

4.2.1 创建歌单

前端校验登录状态，传入歌单名称、描述；后端验证 Token 合法性，根据当前用户 ID 生成全新歌单记录并入库，返回创建结果。

4.2.2 查询歌单列表

后端通过 Token 解析用户 ID，批量查询该用户所有歌单，按创建时间倒序返回列表数据。

4.2.3 编辑歌单

根据歌单 ID 查询歌单归属，判断是否属于当前登录用户；归属合法则更新名称与描述字段，返回最新数据。

4.2.4 删除歌单

权限校验通过后，根据歌单 ID 执行物理删除，删除成功返回操作提示。

4.3 音乐管理模块详细设计

4.3.1 音乐上传

前端选择本地音频文件并填写音乐信息，通过 FormData 提交文件与参数；后端通过 Multer 接收文件并保存至服务器静态资源目录，自动读取音乐时长元数据，生成音乐

数据记录存入数据库。

4.3.2 音乐列表查询

登录用户可查看系统全部已上传音乐资源，后端统一查询所有音乐数据并格式化返回。

4.4 个性化推荐模块详细设计

系统采用基于用户行为的简单偏好统计推荐逻辑：后端统计用户高频播放的歌手、曲风、专辑标签，匹配数据库中相似度最高的音乐与歌单资源，完成个性化推荐输出。无复杂算法，轻量化、高效、适配本系统业务场景。

- 音乐推荐：优先推荐用户高频收听艺术家的同类曲目
- 歌单推荐：根据用户常听曲风匹配对应主题歌单

5 接口详细设计（核心接口）

统一请求前缀：/api，统一返回格式：{code, data, msg}

5.1 用户接口

- POST /api/register：用户注册
- POST /api/login：用户登录，返回 Token

5.2 歌单接口

- POST /api/playlist/create：创建歌单
- GET /api/playlist/list：获取个人歌单列表
- PUT /api/playlist/update：编辑歌单
- DELETE /api/playlist/del：删除歌单

5.3 音乐接口

- POST /api/music/upload：上传音乐文件
- GET /api/music/list：获取全部音乐列表

5.4 推荐接口

- GET /api/recommend/music: 获取推荐音乐
- GET /api/recommend/playlist: 获取推荐歌单

6 页面详细逻辑设计

6.1 登录注册页

区分登录、注册表单，前端完成参数校验，错误实时提示；登录成功保存 Token 并跳转首页，未登录用户无法进入系统主页。

6.2 首页

自动读取用户登录状态，加载个性化推荐音乐、推荐歌单、最近播放记录，动态渲染卡片内容。

6.3 歌单管理页

网格展示所有个人歌单，支持弹窗新建、编辑、删除，操作后实时刷新列表。

6.4 音乐管理页

展示全部音乐资源，提供上传入口，上传成功自动刷新音乐列表。

7 权限与安全设计

- 所有业务接口强制校验 Token，未登录用户禁止访问
- 密码采用加密存储，数据库不保存明文密码
- 歌单、音乐资源严格校验归属用户，防止越权操作
- 文件上传限制文件类型，仅允许音频格式上传，规避恶意文件风险

8 系统扩展性设计

系统采用模块化分层开发，接口、数据模型、页面组件完全解耦，可后续快速拓展：歌单收藏、歌单分享、音乐播放进度记录、用户偏好标签细化、智能分类、评论系统等功能。

9 总结

本文档从数据库结构、模块内部逻辑、接口规范、页面交互、权限安全等维度完成了系统详细设计，明确了所有功能的具体实现方式。系统结构清晰、逻辑严谨、代码可落地性强，完全满足音乐个性化管理的业务需求，可直接作为项目开发、测试、结题答辩的标准技术文档。

|（注：部分内容可能由 AI 生成）